

# Chapter 3

## Memory Hierarchy



# Four Questions for Cache Designers

**Caching** is a general concept used in processors, operating systems, file systems, and applications.

There are **Four Questions** for Cache/Memory Hierarchy Designers

- **Q1:** Where can a block be placed in **the upper level/main memory**?  
*(Block placement)*
  - Fully Associative, Set Associative, Direct Mapped
- **Q2:** How is a block found if it is in **the upper level/main memory**?  
*(Block identification)*
  - Tag/Block
- **Q3:** Which block should be replaced on a **Cache/main memory** miss?  
*(Block replacement)*
  - Random, LRU, FIFO
- **Q4:** What happens on a write?  
*(Write strategy)*
  - Write Back or Write Through (with Write Buffer)



## Q4: Write Strategy-- Write Hit

- **write-through**
  - On a write hit, data is written **simultaneously to both the cache and the main memory**.
  - Ensures strong data consistency; every write reaches main memory immediately.
  - Cache control bit: only a valid bit
  - memory always have latest data
  - **Typical Use Case:** Real-time systems (e.g., flight control), where data integrity is critical .



## Q4: Write Strategy-- Write Hit

- **write-back**

- On a write hit, data is written **only to the cache**.
- The main memory is updated **only when the modified (dirty) block is evicted** from the cache.
- Uses a **dirty bit** to track modifications; reduces memory bandwidth usage.
- Cache control bits: both valid and dirty bits.
- **Typical Use Case:** General-purpose processors (e.g., desktop CPUs), where performance is prioritized .



## Q4: Write Strategy

- **Dirty Bit**

- A status bit associated with a cache line that indicates whether the data in the cache has been **modified but not yet written back** to main memory.
- Essential for **Write-Back** policy to determine if a write-back is needed during eviction.
- **Typical Use Case:** All modern CPUs using write-back caching (e.g., Intel Core series) .



## Q4: Write Strategy-- Write miss

- **Write allocate**

- On a **write miss**, the system **loads the missed block from main memory into the cache**, then performs the write operation in the cache.
- Exploits spatial and temporal locality for future accesses.
- **Typical Use Case**: Paired with **Write-Back** in L2/L3 caches to optimize repeated writes.



## Q4: Write Strategy-- Write miss

- **no write allocate** (Write around)
  - On a **write miss**, data is written **directly to main memory without loading the block into the cache**.
  - It is not stored in the cache. Prevents cache pollution from one-time or infrequent writes.
  - **Typical Use Case:** Logging systems or streaming data, where written data is rarely read again .



## Q4: Write Strategy-- Write miss

- **write-back + write-allocate**

- On a write miss, the block containing the target address (e.g., 0x1000) is first **loaded from main memory into the cache**.
- The data is then modified within the cache, and the dirty bit is set.
- The modified data is written back to main memory only when this cache block is eventually replaced.

- **write-through + no write-allocate**

- On a write miss, the data is written directly to the main memory **without loading the corresponding block into the cache**.
- Since no copy exists in the cache, any subsequent read access to this address will result in a cache miss, requiring another access to main memory.



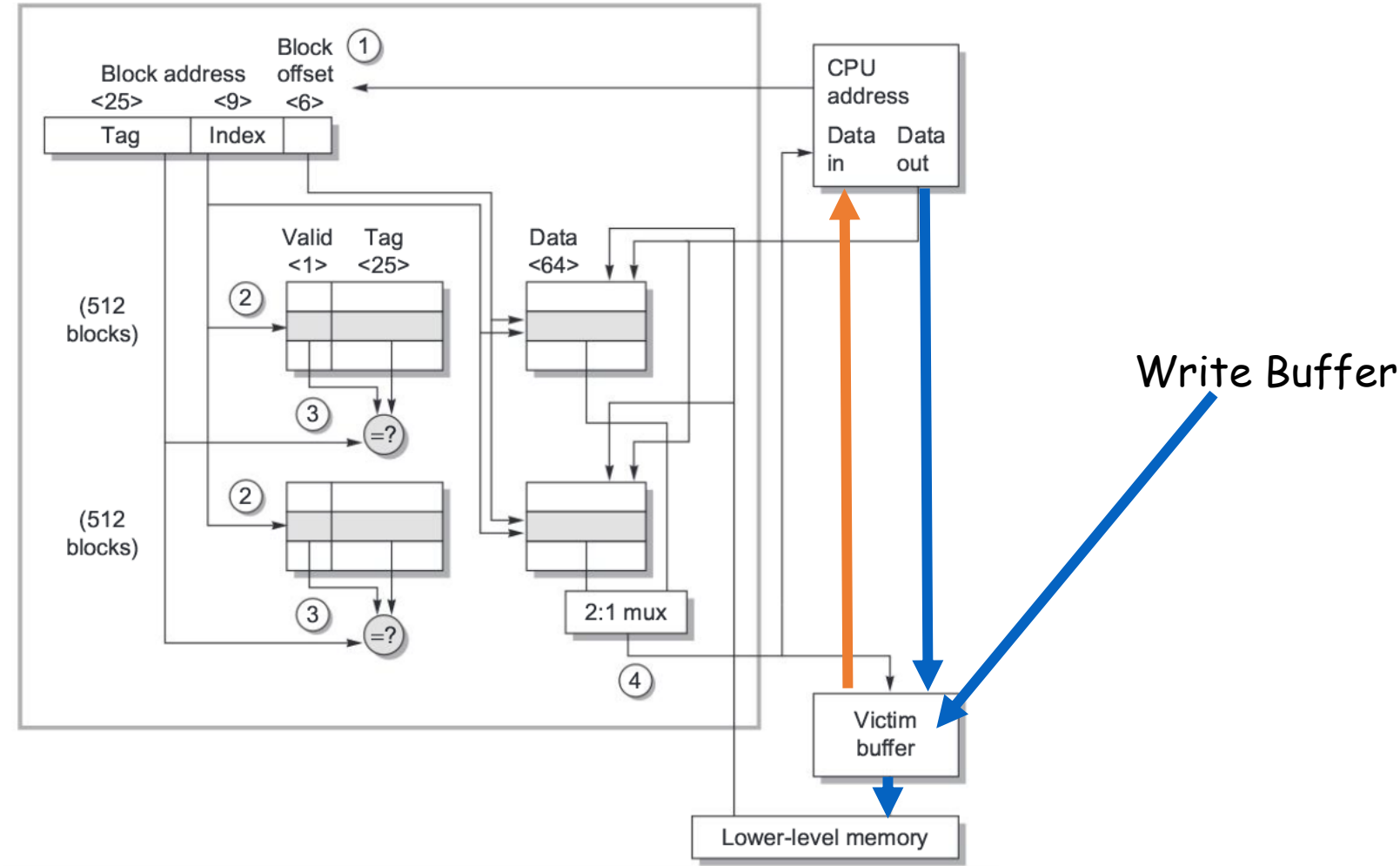


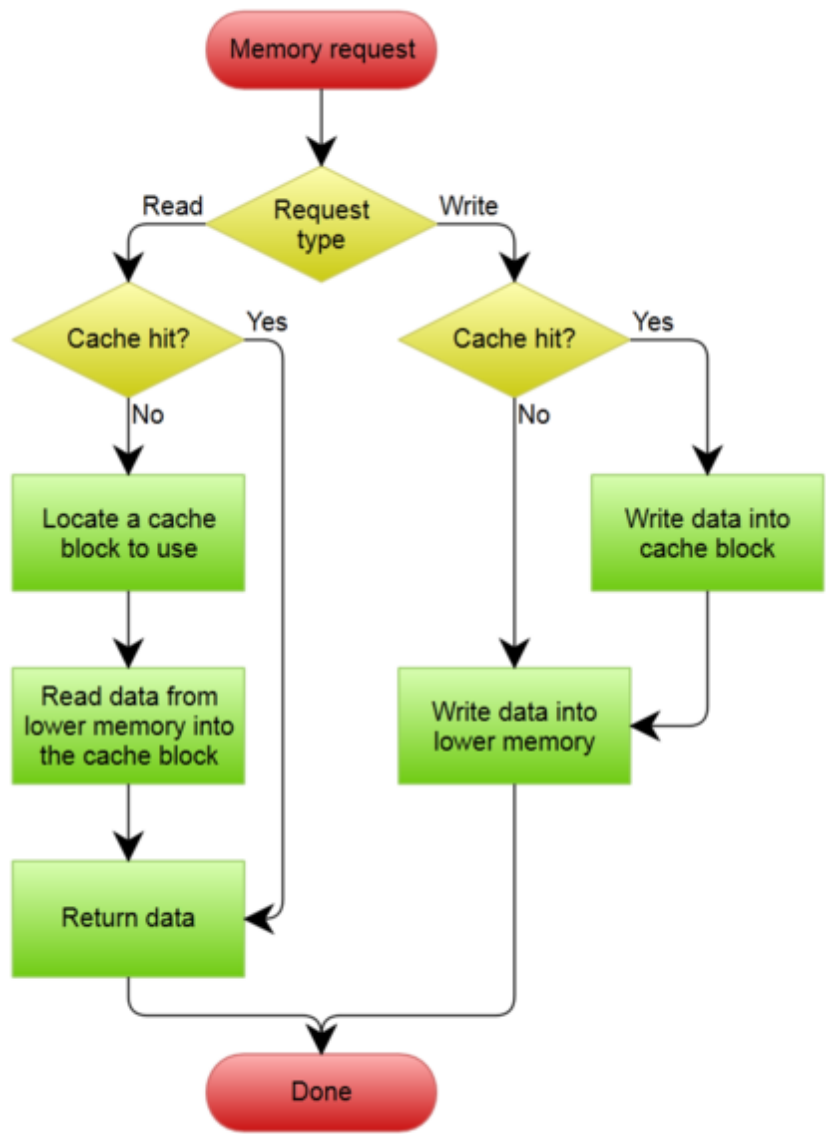
# Write stall & Write buffer

- **Write stall**
  - When the CPU must wait for writes to complete during write through
- **Write buffer** (for Write-Through Optimization)
  - To avoid stalling on writes, many CPUs use a write buffer.
  - A small buffer that **temporarily holds write-through data**, allowing the CPU to continue execution without waiting for memory writes to complete.
  - Mitigates performance penalty of Write-Through policy.
  - This buffer helps when writes are clustered.
  - It does not entirely eliminate stalls since it is possible for the buffer to fill if the burst is larger than the buffer.

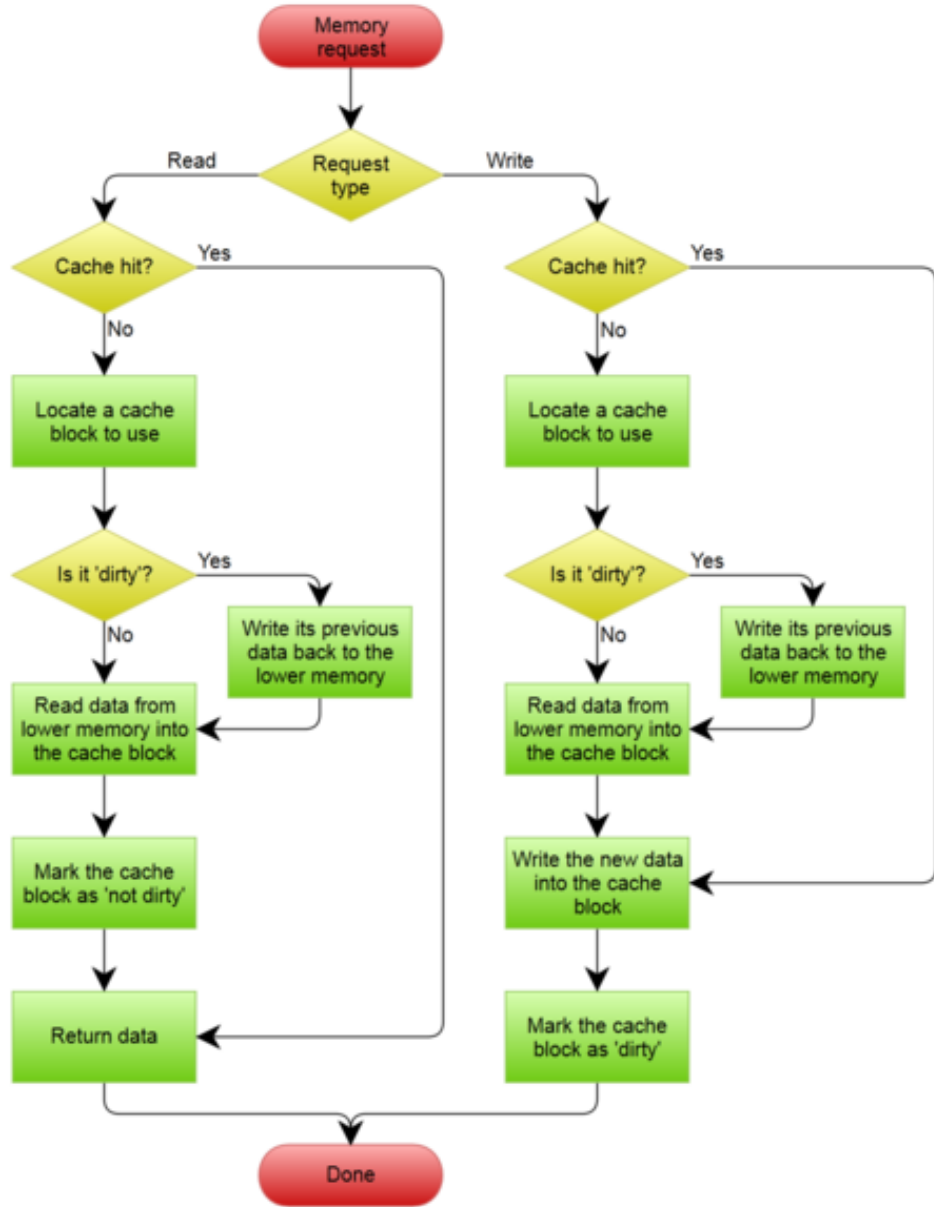


# Write buffer





A Write-Through cache with No-Write Allocation  
Read allocate



A Write-Back cache with Write Allocation  
Read allocate



# Example

- (1) Which strategy updates both the cache and main memory on a write hit?
  - A. Write-Back
  - B. Write-Through
  - C. No-Write-Allocate
  - D. Replacement Algorithm
  
- (2) What happens if a cache block modified under the Write-Back policy is lost due to power failure before being evicted?
  - A. Data is not lost, as main memory was already updated
  - B. Data is lost, since the update existed only in the cache
  - C. The system automatically flushes the data before shutdown
  - D. Cache prevents power loss from occurring

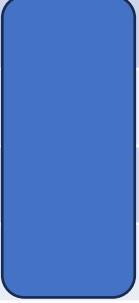


# Example

(3) Consider two cache write strategies:

- **Strategy A:** Write-Back + Write-Allocate
- **Strategy B:** Write-Through + No-Write-Allocate

Analyze their behavior on a **write miss** (target address not in cache):

| Action                                         | Strategy A                                                                           | Strategy B                                                                           |
|------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Load block from main memory into cache         |  |  |
| Update main memory immediately                 |                                                                                      |                                                                                      |
| Update cache                                   |                                                                                      |                                                                                      |
| Subsequent read the same address hits in cache |                                                                                      |                                                                                      |

(4) Match the following applications with the most appropriate write policy combination:

- Real-time control system (e.g., aircraft flight controller)
- General-purpose desktop computer user process cache
- High-frequency trading server's temporary data cache



# Example

- (5) Assume a fully associative write-back cache with many cache entries that starts empty below is a sequence of five memory operations(the address is in square brackets):

```
1  write Mem[100];  
2  write Mem[100];  
3  read  Mem[200];  
4  write Mem[200];  
5  write Mem[100];
```

What are the number of hits and misses when using  
**no-write allocate** versus **write allocate**?



# Example

- Assume a fully associative write-back cache with many cache entries that starts empty below is a sequence of five memory operations(the address is in square brackets):

|                   | no-write allocate | write allocate |
|-------------------|-------------------|----------------|
| 1 write Mem[100]; | miss              | miss           |
| 2 write Mem[100]; | miss              | hit            |
| 3 read Mem[200];  | miss              | miss           |
| 4 write Mem[200]; | hit               | hit            |
| 5 write Mem[100]; | miss              | hit            |

What are the number of hits and misses when using **no-write allocate** versus **write allocate**?



# Summary

## Read Cache

Read through

Read allocate

## Write Cache

hit

*Write-through*

write is done synchronously  
both to the cache and to the  
backing store

*Write-back*

writing is done only to  
the cache

miss

*Write allocate*

data at the missed-write location is  
loaded to cache, followed by a write-  
hit operation.

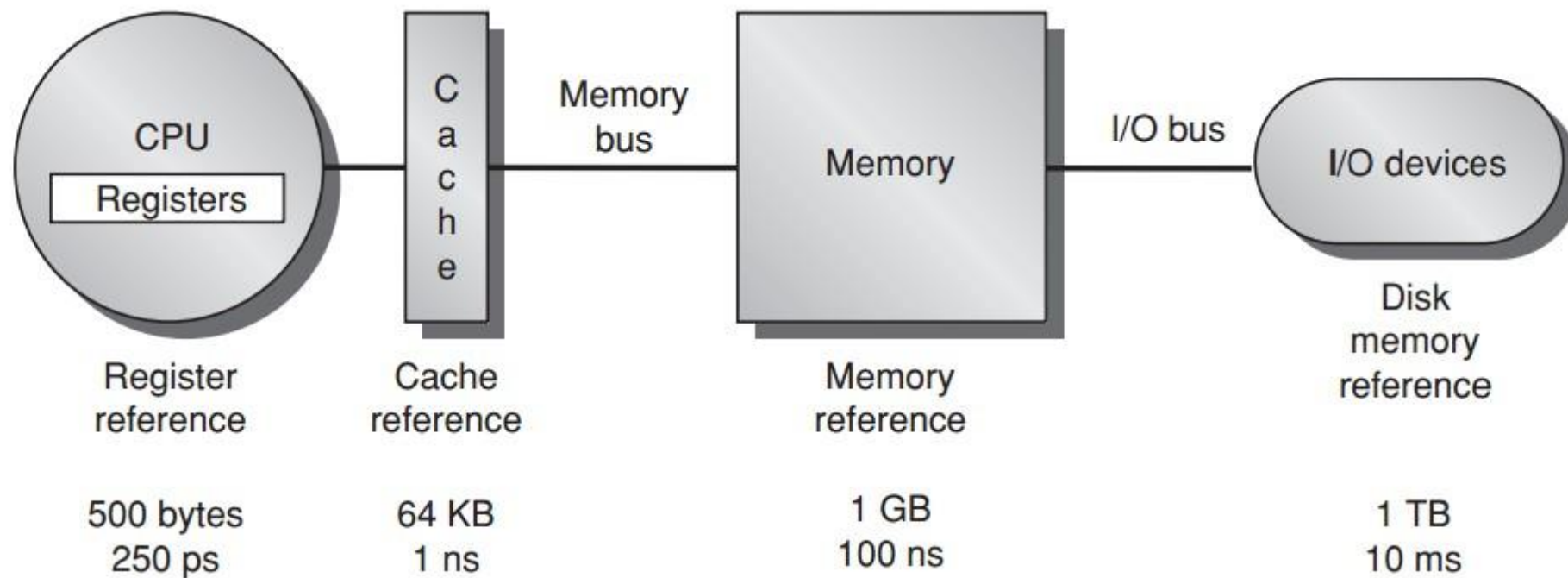
*No-write allocate*

data at the missed-write location is not  
loaded to cache, and is written directly  
to the backing store

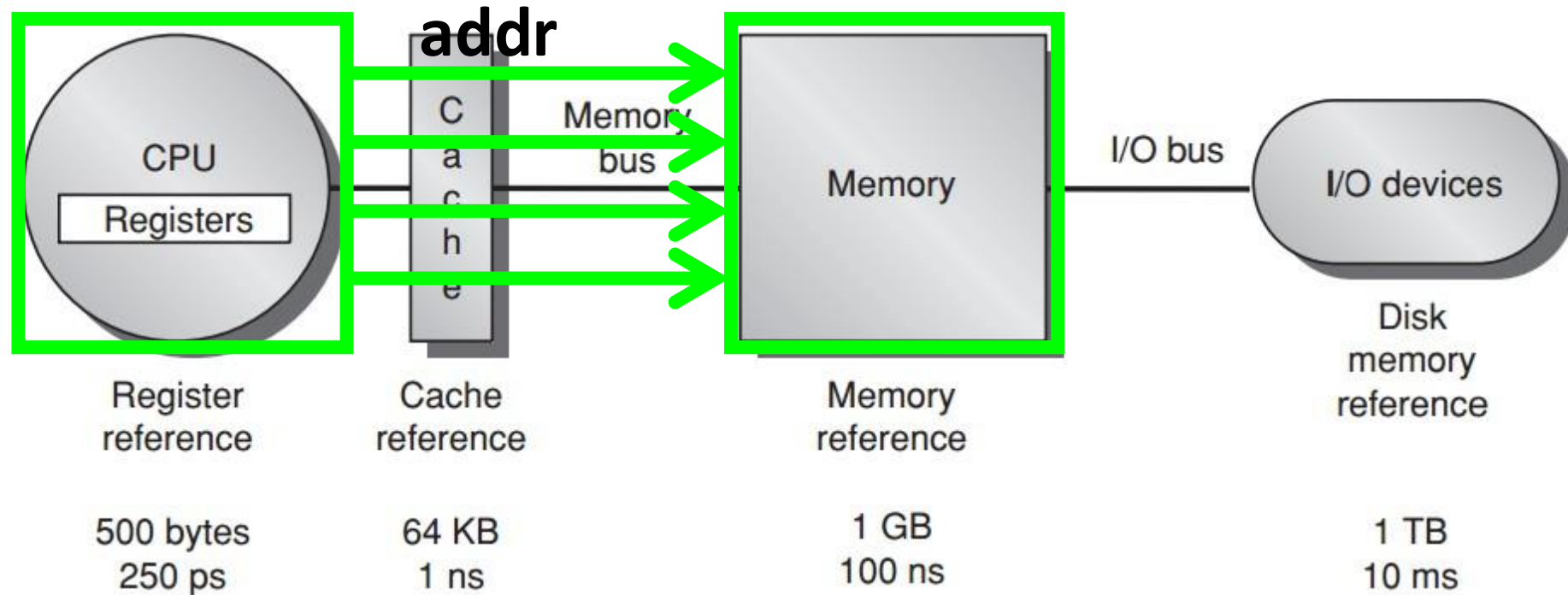




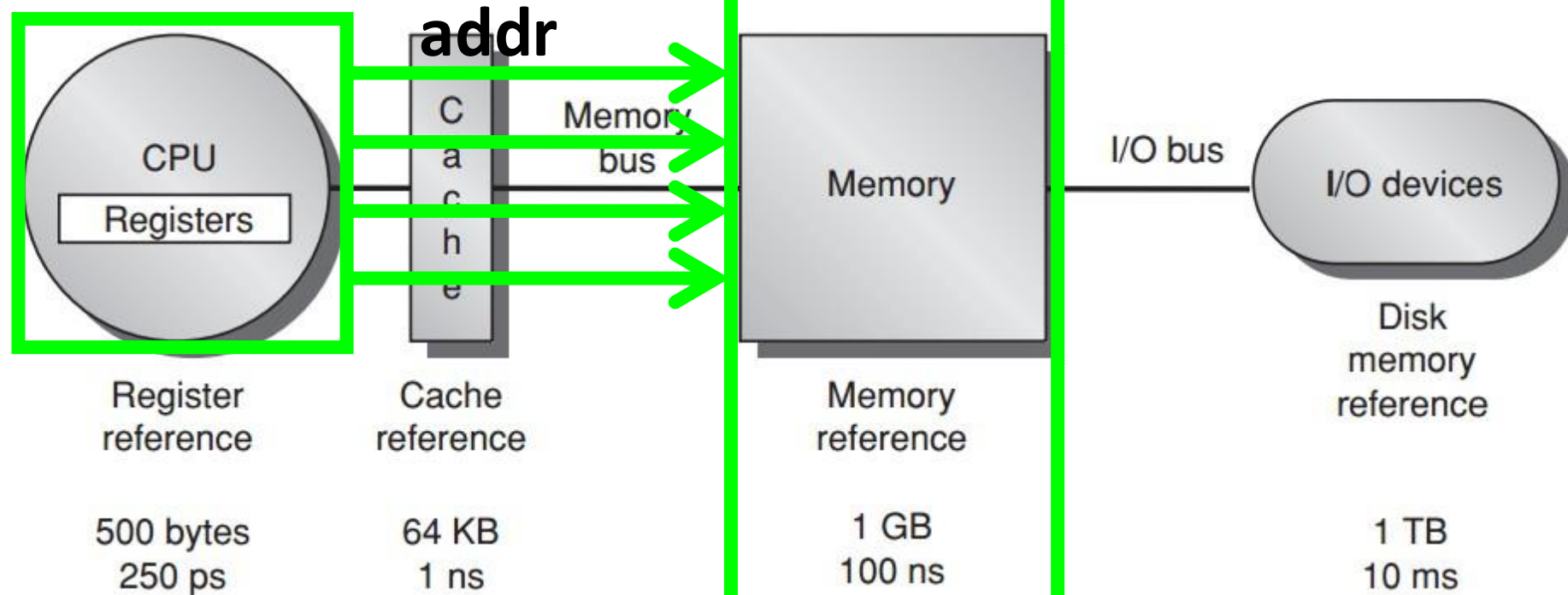
# Memory Hierarchy



# Memory Hierarchy

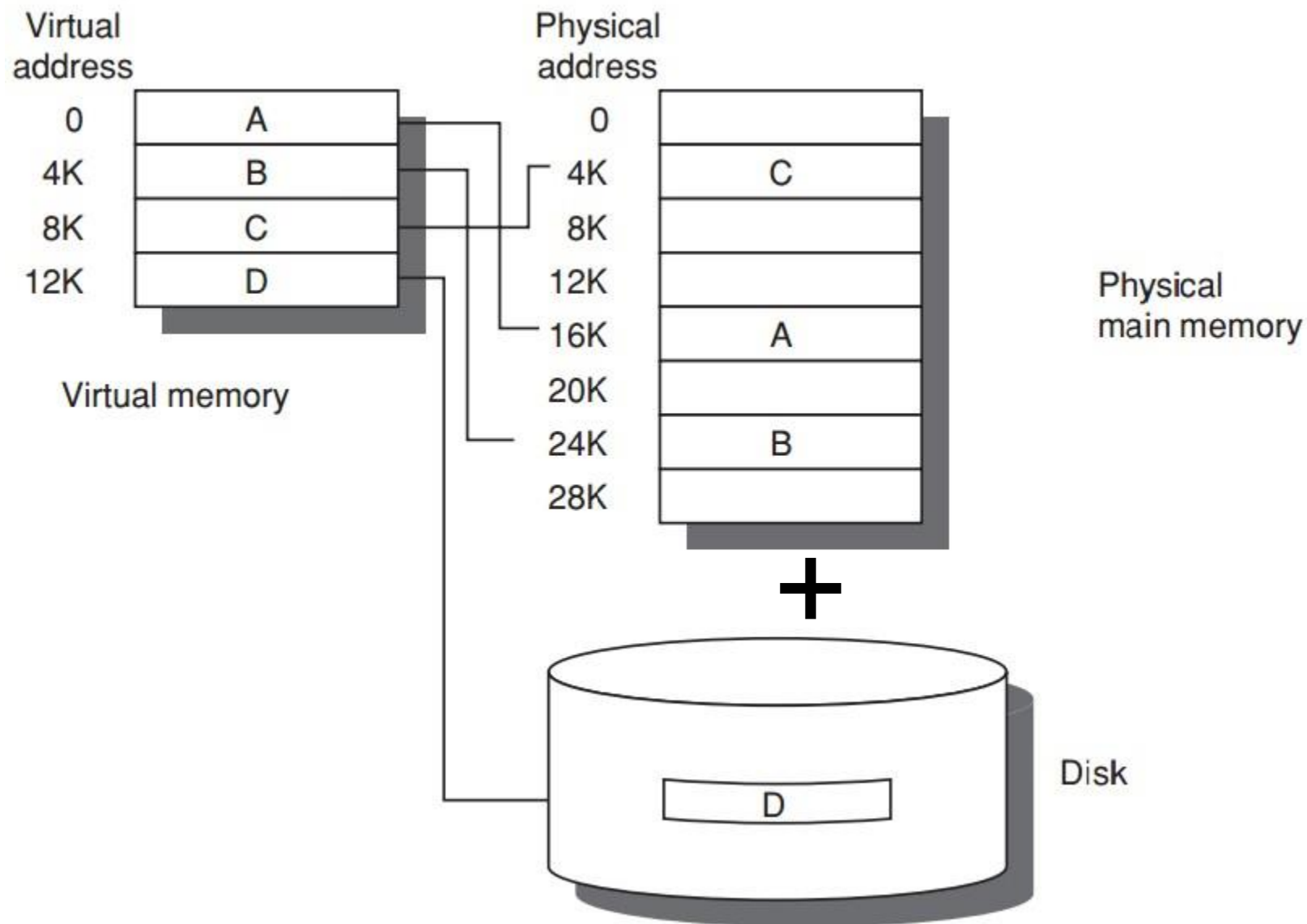


# Memory Hierarchy

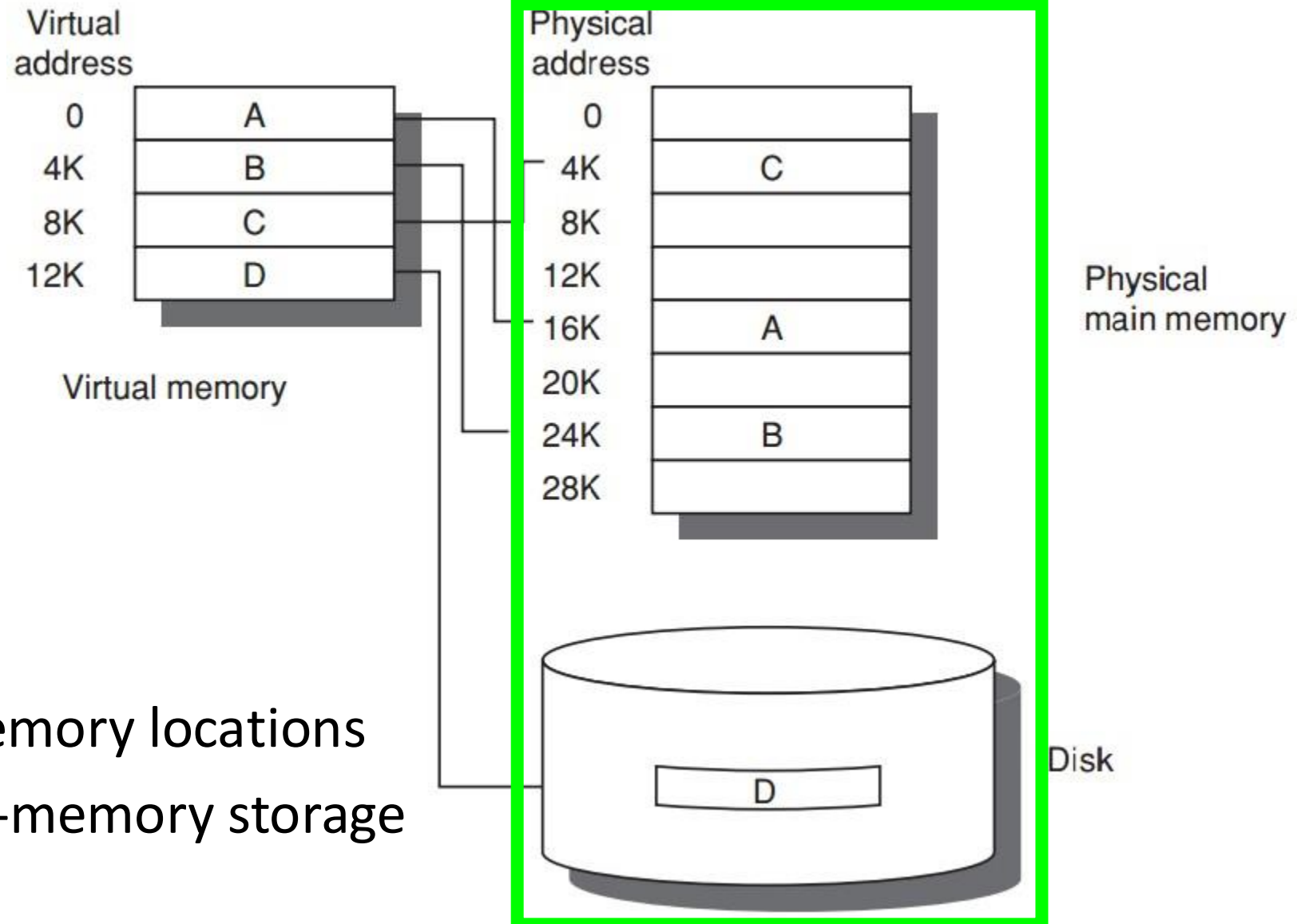


***Larger memory for more processes?***

# Virtual Memory



# Virtual Memory



Program **uses**

- discontinuous memory locations
- + secondary/non-memory storage

# How memory hierarchy works?

Questions:

- Q1. *Where can a block be placed in main memory?*
- Q2. *How is a block found if it is in main memory?*
- Q3. *Which block should be replaced on a virtual memory miss?*
- Q4. *What happens on a write?*